

M3 — Integración numérica y costo computacional

Cómo se resuelve una EDO en la computadora, y qué cuesta hacerlo

Proyecto Wilson–Cowan — Iniciación a la I+D ITBA

12 de julio de 2026

Índice

1. Cómo se resuelve una EDO en la computadora	1
2. Métodos: Euler, RK4 y paso adaptativo	1
3. Error local, error global y orden	2
4. NFE: el costo como número de evaluaciones de f	3
5. Rigidez, paso estable y la salvedad de los estímulos conmutados	3
6. Por qué el integrador tiene que ser diferenciable	4

Antes de empezar

Este documento requiere **M2 (EDOs y sistemas dinámicos)**: damos por conocido qué es una EDO $\dot{\mathbf{x}} = f(\mathbf{x}, t)$, qué es un flujo, el Jacobiano $\partial f / \partial \mathbf{x}$ y sus autovalores. Acá pasamos de “la EDO tiene una solución” a “cómo obtengo esa solución en la computadora, y cuánto me cuesta”. Es la maquinaria numérica sobre la que se apoya toda la identificación del proyecto: cada vez que entrenamos la Neural ODE para recuperar los 10 parámetros de Wilson–Cowan (WC), por dentro hay un integrador corriendo miles de veces. Entender su costo es entender de dónde sale el tiempo de cómputo.

1. Cómo se resuelve una EDO en la computadora

Una EDO $\dot{\mathbf{x}} = f(\mathbf{x}, t)$ dice cuál es la *pendiente* del estado en cada punto, pero no da la trayectoria $\mathbf{x}(t)$ en forma cerrada salvo casos triviales. WC, con sus dos poblaciones acopladas y sigmoideas, no tiene solución analítica. La computadora la construye **a mano, en pasos discretos**.

La idea. Partimos de una condición inicial $\mathbf{x}(t_0) = \mathbf{x}_0$ conocida y de una grilla temporal $t_0 < t_1 < t_2 < \dots$ con paso $\Delta t = t_{n+1} - t_n$. En cada paso usamos la información de la pendiente f para estimar hacia dónde se mueve el estado y avanzar un cachito:

$$\mathbf{x}_{n+1} = \mathbf{x}_n + (\text{incremento basado en } f). \quad (1)$$

El resultado es una *secuencia* de puntos $\{\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2, \dots\}$ que aproxima la trayectoria continua. La calidad de la aproximación depende de dos cosas: cuán chico es Δt y cuán inteligente es la forma de estimar el incremento. Esos dos ejes definen a los *métodos de integración*.

Observación 1. En el proyecto, “resolver WC” aparece por todos lados: para generar los datos sintéticos, para simular el modelo candidato dentro del entrenamiento (el rollout de la Neural ODE) y para correr la planta en el lazo de control (ver $P4$, `neural_ode`). Todas esas veces se está avanzando en pasos Δt .

2. Métodos: Euler, RK4 y paso adaptativo

La familia estándar es la de **Runge–Kutta**: métodos que estiman el incremento evaluando f en uno o varios puntos dentro del paso y promediando esas pendientes [1].

Euler (orden 1). El más simple: usa la pendiente al principio del paso y avanza recto.

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \Delta t f(\mathbf{x}_n, t_n). \quad (2)$$

Una sola evaluación de f por paso. Es barato pero *tosco*: como extrapola con una recta, en una curva se va quedando “por fuera” y el error se acumula. Para lograr precisión decente hay que usar Δt muy chico, y ahí lo barato por paso se paga en muchísimos pasos.

Runge–Kutta 4 (RK4, orden 4). El caballito de batalla, y el que usa el proyecto. En cada paso evalúa f en **cuatro puntos** (el inicio, dos estimaciones en el medio, y el final tentativo) y los combina con un promedio ponderado:

$$\begin{aligned} k_1 &= f(\mathbf{x}_n, t_n), & k_2 &= f\left(\mathbf{x}_n + \frac{\Delta t}{2} k_1, t_n + \frac{\Delta t}{2}\right), \\ k_3 &= f\left(\mathbf{x}_n + \frac{\Delta t}{2} k_2, t_n + \frac{\Delta t}{2}\right), & k_4 &= f(\mathbf{x}_n + \Delta t k_3, t_n + \Delta t), \\ \mathbf{x}_{n+1} &= \mathbf{x}_n + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4). \end{aligned} \quad (3)$$

Cuesta $4\times$ lo de Euler por paso, pero es tanto más preciso que compensa de sobra: con el *mismo* Δt el error es órdenes de magnitud menor. La Figura 1 muestra esta comparación sobre WC.

Paso fijo vs. adaptativo. RK4 tal como está usa *paso fijo*: el mismo Δt de punta a punta. Esto es simple y predecible, pero derrocha esfuerzo en tramos donde la dinámica es suave y puede quedarse corto donde es abrupta. Los métodos de **paso adaptativo**, como *Dormand–Prince* (el `ode45` de Matlab, también llamado RK45), *estiman el error local* en cada paso comparando dos fórmulas de distinto orden y ajustan Δt automáticamente: pasos grandes donde la solución es suave, chicos donde hay que resolver detalle fino. Ganan precisión por unidad de cómputo en problemas exigentes, a cambio de perder la regularidad de la grilla, que — como veremos — el proyecto necesita.

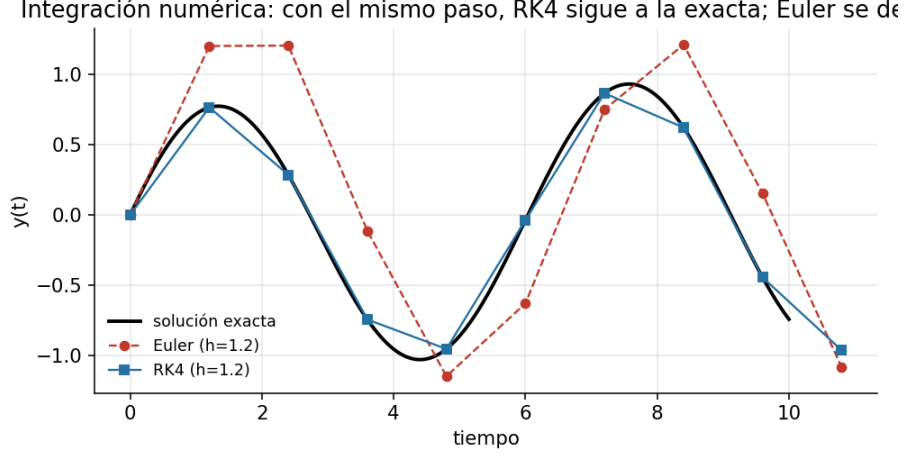


Figura 1: Integración de Wilson–Cowan con el mismo paso Δt : solución de referencia (exacta) frente a Euler y RK4. Euler se despega visiblemente de la curva verdadera; RK4 la sigue casi sin error. El costo por paso de RK4 es $4\times$ el de Euler, pero la ganancia de precisión es mucho mayor que $4\times$.

3. Error local, error global y orden

Definición 1 (Error local y global). El **error local** (o de truncamiento) es el que se comete en un paso, suponiendo que se arrancó desde el valor exacto. El **error global** es el acumulado a lo largo de toda la trayectoria, tras muchos pasos.

Definición 2 (Orden de un método). Un método es de **orden** p si su error global escala como $O(\Delta t^p)$. Equivalentemente, su error local es $O(\Delta t^{p+1})$ (se pierde un orden al acumular sobre $\sim 1/\Delta t$ pasos).

La consecuencia práctica es lo que importa: si **reduzco Δt a la mitad**,

- con **Euler** (orden 1) el error global cae $\sim 2\times$;
- con **RK4** (orden 4) el error global cae $\sim 2^4 = 16\times$.

Por eso el orden es tan valioso: un método de orden alto llega a la misma precisión con pasos mucho más grandes, es decir con *menos pasos totales*. Esta es la palanca central de la próxima sección.

4. NFE: el costo como número de evaluaciones de f

¿Cómo medir el costo de integrar sin depender del hardware, del lenguaje ni de cuán ocupada está la máquina? La medida estándar en la literatura de Neural ODEs [2] es el **NFE** (*number of function evaluations*): cuántas veces se evalúa la función f en total. Es una métrica limpia porque cada evaluación de f es la operación cara (en el gray-box del proyecto, calcular las sigmoideas y el acoplamiento de WC), y el resto es aritmética barata.

Con **RK4 de paso fijo** el NFE es *determinista*: como cada paso hace exactamente 4 evaluaciones,

$$\boxed{\text{NFE} = 4 \times (\text{número de pasos})} \quad (4)$$

y el número de pasos es $\approx T/\Delta t$ para un horizonte T . Escalando al entrenamiento completo:

$$\text{NFE}_{\text{total}} = 4 \times (\text{pasos por trayectoria}) \times (\text{trayectorias}) \times (\text{épocas}). \quad (5)$$

De acá sale la conclusión operativa del proyecto: en identificación *pura* sobre datos de WC corremos con la corrección neuronal apagada, así que el “modelo” son apenas ~ 10 escalares y

su costo es despreciable; **el costo real vive en la integración**, no en el tamaño de la red. Duplicar Δt divide el NFE por dos, de forma directa y medible (ver M5 para el detalle de la arquitectura).

5. Rigidez, paso estable y la salvedad de los estímulos conmutados

Si bajar Δt mejora la precisión y subirlo abarata el cómputo, ¿qué impide poner Δt enorme? Dos límites: la *precisión* que exijamos y, sobre todo, la **rigidez** (*stiffness*) del sistema.

Rigidez. Un sistema es rígido cuando conviven escalas de tiempo muy dispares: modos que decaen rapidísimo junto a modos lentos. Se cuantifica con los autovalores del Jacobiano de estado $\partial f / \partial \mathbf{x}$: la *razón de rigidez* $|\lambda_{\text{máx}}| / |\lambda_{\text{mín}}|$ mide ese contraste. Cuanto más rígido, más chico debe ser Δt para que un método explícito como RK4 sea *estable* (si no, la solución numérica oscila o diverge aunque la real sea mansa) [3]. Nótese el paralelo con M4 (Fisher+SVD): allá el espectro de una sensibilidad a *parámetros* predice identificabilidad; acá el espectro del Jacobiano de *estado* predice el costo de integración. Misma idea, distinta derivada.

WC es poco rígido. Sus dos constantes de tiempo ($\tau_e = 1$, $\tau_i = 2$) son del mismo orden: no hay separación brutal de escalas. Por eso WC **admite pasos grandes**: bajar la resolución temporal reduce el costo casi sin perder calidad *en estímulos suaves*. Medido en el proyecto: se pasó de $\Delta t = 0,05$ a $\Delta t = 0,2$ (4× menos pasos), con el tiempo por época cayendo de ~ 510 ms a ~ 140 ms, sin degradar la identificación. Se evaluó además un solver adaptativo: *no aporta*, precisamente porque WC no es rígido y no hay tramos que justifiquen refinar el paso.

Salvedad: estímulos conmutados. Acá está el hallazgo fino del proyecto, y por qué el paso grande no es gratis siempre. Los estímulos $P(t), Q(t)$ que buscamos — pensados para ser realizables con optogenética — suelen ser **conmutados**: pulsos, escalones, *on/off*. Entre muestras, el estímulo se mantiene constante mediante un *retenedor de orden cero* (ZOH). En cada **conmutación** (cada flanco del estímulo) hay una discontinuidad, y el ZOH la ubica sobre la grilla con un error que no mejora con el orden del método: es de orden bajo por construcción. Con Δt grande esos flancos quedan mal alineados en el tiempo y el error se acumula justo donde la dinámica lleva la información que más excita al sistema. Resultado medido: con estímulos conmutados a $\Delta t \approx 0,2$, la identificación **colapsa** (el error de w_{II} trepa a 250–400 %). La regla práctica queda entonces partida: Δt **grande solo para estímulos suaves / de banda limitada** (p.ej. un *chirp*); Δt **fino cuando el estímulo conmuta**. La Figura 2 muestra este trade-off medido.

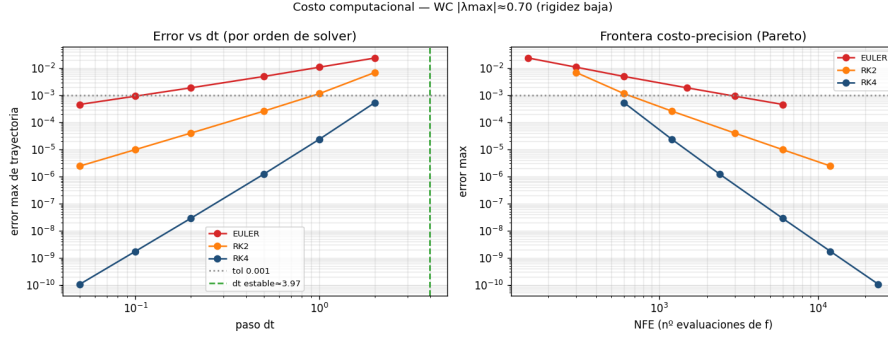


Figura 2: Costo computacional vs. paso de integración Δt (figura del proyecto). Subir Δt baja monótonamente el costo (menos pasos, menos NFE), pero la calidad se sostiene solo mientras el estímulo sea suave; con estímulos conmutados hay un Δt máximo por debajo del cual hay que quedarse para no romper la identificación.

6. Por qué el integrador tiene que ser diferenciable

Hasta acá integramos para *simular*. Pero en el proyecto integrar es un medio para un fin mayor: **entrenar**. La Neural ODE aprende los parámetros θ minimizando el error entre la trayectoria simulada y la medida,

$$\mathcal{L}(\theta) = \left\| \text{rollout}(f_{\theta}, \mathbf{x}_0, P, Q, \Delta t) - \mathbf{y}_{\text{medido}} \right\|^2, \quad (6)$$

y para minimizar por descenso de gradiente necesitamos $\partial \mathcal{L} / \partial \theta$. El problema: $\mathcal{L}$ no depende de θ directamente, sino *a través del integrador*. El gradiente tiene que **atravesar todo el solver**.

Acá es donde (3) paga: RK4 es una *composición de operaciones diferenciables* (sumas, productos, evaluaciones de f , que a su vez son sigmoideas suaves). Entonces la regla de la cadena — la *backpropagation* — puede recorrer los cientos de pasos del *rollout* hacia atrás y entregar el gradiente exacto respecto de θ . Un integrador “de caja negra” (una rutina compilada que solo devuelve números) rompería esta cadena; por eso el proyecto usa un integrador **diferenciable**, escrito en un framework de autograd. Esto es lo que convierte a la integración numérica de una mera herramienta de simulación en el *motor* del aprendizaje, y es el punto de contacto con M5, donde se desarrolla la arquitectura Neural ODE / PINN y el uso de *multiple shooting* para que ese gradiente a lo largo de trayectorias largas no explote.

Referencias

- [1] E. Hairer, S. P. Nørsett y G. Wanner, *Solving Ordinary Differential Equations I: Nonstiff Problems*, 2.^a ed., Springer, 1993.
- [2] R. T. Q. Chen, Y. Rubanova, J. Bettencourt y D. Duvenaud, “Neural Ordinary Differential Equations”, *NeurIPS*, 2018.
- [3] S. Kim *et al.*, “Stiff neural ordinary differential equations”, *Chaos*, 31(9):093122, 2021.